

Verify and troubleshoot a local Camunda 8 environment

Technical writing sample | Daniel Fowler

Use this guide to start the lightweight Camunda 8.9 Self-Managed distribution with Docker Compose and confirm that the Orchestration Cluster is ready to use.

By the end, you will be able to:

- validate the Compose configuration before startup;
- start Camunda in the background;
- confirm the containers and web applications are available;
- test the Orchestration Cluster REST API; and
- identify the cause of common local startup failures.

Local development only

Camunda's Docker Compose distribution is intended for development and evaluation. Use Kubernetes with Helm for production deployments.

Before you begin

You need:

- Docker 20.10.16 or later;
- Docker Compose 1.27.0 or later, using the `docker compose` command;
- a terminal with access to Docker; and
- the Camunda 8.9 Docker Compose distribution, downloaded and extracted.

The default `docker-compose.yaml` file starts the lightweight environment. It includes the Orchestration Cluster, Connectors, and Elasticsearch. This is the best option for most local development work.

1. Confirm Docker is available

Run:

```
docker --version
docker compose version
```

Both commands should return version information. If `docker compose` is not recognised, update Docker Desktop or install the current Docker Compose plugin before continuing.

2. Validate the Compose configuration

Open a terminal in the extracted directory that contains `docker-compose.yaml`, then run:

```
docker compose config --quiet
```

A successful validation produces no output and returns control to the terminal.

If the command reports a missing variable or invalid attribute, do not start the environment yet. Check that:

1. you are in the directory containing the Compose file;
2. any accompanying `.env` file is present; and
3. you are using `docker compose`, rather than the legacy `docker-compose` command.

To inspect the fully resolved configuration, run:

```
docker compose config
```

This is useful when an environment variable or port mapping is not behaving as expected.

3. Start Camunda 8

Start the lightweight environment in detached mode:

```
docker compose up -d
```

Detached mode keeps the containers running while leaving the terminal available. The first startup can take several minutes because Docker may need to download the required images.

Check the state of every service:

```
docker compose ps --all
```

Look for services that are running or becoming healthy. An exited or repeatedly restarting service needs attention before the environment will work correctly.

Tip

Service names can change between releases. Use the service names shown by `docker compose ps` when running the log commands below.

4. Verify the web applications

When the containers are ready, open the following addresses:

Component	Address	Purpose
Operate	http://localhost:8080/operate	Monitor and troubleshoot process instances
Tasklist	http://localhost:8080/tasklist	Work with user tasks
Admin	http://localhost:8080/admin	Manage users and permissions in the lightweight environment

Use the default credentials:

- **Username:** demo
- **Password:** demo

A page that loads but remains unavailable may indicate that the container has started but the application is still initialising. Wait briefly, then check its logs rather than repeatedly restarting the whole environment.

5. Test the REST API

The lightweight configuration exposes the Orchestration Cluster REST API at <http://localhost:8080/v2>.

Request the cluster topology:

```
curl http://localhost:8080/v2/topology
```

A working cluster returns a JSON response describing its brokers, partitions, and cluster size. This confirms that the API is reachable and the Orchestration Cluster can respond to requests.

If the web applications load but this request fails, check the HTTP status and response body:

```
curl -i http://localhost:8080/v2/topology
```

The headers help distinguish a connection problem from an authentication or server error.

Troubleshooting

A service has exited

List all containers, including stopped ones:

```
docker compose ps --all
```

Then view the last 100 log lines for the affected service:

```
docker compose logs --tail 100 <service-name>
```

Replace <service-name> with the name shown in the SERVICE column. Read from the first error, not only the final line: later failures are often consequences of an earlier configuration or dependency problem.

A service keeps restarting

Follow its logs while it restarts:

```
docker compose logs --follow <service-name>
```

Press Ctrl+C to stop following the output. The containers will continue running.

Common causes include:

- another application already using a required host port;
- insufficient resources assigned to Docker;
- an incomplete image download; or
- a missing or incorrectly resolved environment variable.

Docker reports that a port is already allocated

The startup output normally identifies the conflicting port. Stop the application using that port, or change the host-side port mapping in the Compose configuration.

After editing the file, validate it again:

```
docker compose config --quiet
```

Then apply the change:

```
docker compose up -d
```

The API returns 401 Unauthorized

Confirm which Compose configuration you started. In Camunda 8.9, the default lightweight configuration exposes the local REST and gRPC APIs without authentication. The full configuration uses OAuth for API access.

If you started `docker-compose-full.yaml`, use the authentication settings supplied with that configuration rather than treating the 401 response as a connectivity failure.

You need a clean restart

To stop the environment while retaining its volumes, run:

```
docker compose down
```

To stop the environment and delete its persisted data, run:

```
docker compose down -v
```

The `-v` option removes volumes, including process data and local users. Use it only when you deliberately want to reset the environment.

Before escalating a problem

Collect enough information for another developer or support engineer to reproduce the issue:

- the Camunda distribution version and Compose file you used;
- the output of `docker --version` and `docker compose version`;
- the output of `docker compose ps --all`;
- the last 100 log lines from each affected service;
- the exact command that failed and its complete error message; and
- any changes you made to the Compose file or `.env` values.

Protect sensitive information

Remove passwords, tokens, client secrets, and other sensitive values before sharing logs or configuration. A short, reproducible report is usually more useful than a large, unfiltered log file.

Result

You now have a repeatable way to start a local Camunda 8.9 environment, validate its configuration, confirm the user interfaces and REST API are available, and investigate failed services without immediately destroying the environment.

Reference sources

- [Camunda 8.9: Developer quickstart with Docker Compose](#)
- [Camunda 8.9: Configure Docker Compose environments](#)
- [Camunda 8.9: Orchestration Cluster REST API](#)
- [Docker CLI: docker compose config](#)
- [Docker CLI: docker compose ps](#)
- [Docker CLI: docker compose logs](#)